

Received 25 February 2025, accepted 2 May 2025, date of publication 7 May 2025, date of current version 14 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3567759

RESEARCH ARTICLE

Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?

IOANNA MANDILARA^{ID}, CHRISTINA MARIA ANDRONA^{ID}, ELENI FOTOPOULOU^{ID},
ANASTASIOS ZAFEIROPOULOS^{ID}, (Member, IEEE),
AND SYMEON PAPAVALASSIOU^{ID}, (Senior Member, IEEE)

Institute of Communication and Computer Systems, National Technical University of Athens, 157 80 Athens, Greece

Corresponding author: Anastasios Zafeiropoulos (tzafeir@cn.ntua.gr)

This work was supported by European Union's Horizon 2020 Research and Innovation Program under Grant 101037424.

ABSTRACT The integration of Knowledge Graphs (KGs) with Question Answering (QA) systems is transforming the landscape of Artificial Intelligence (AI). Through the combination of these technologies, novel features can be provided for the translation of questions in natural language into database queries. Even if a lot of work is emerging in this domain, this is not the case when we refer to translation of text to Cypher queries, where Cypher is one of the dominant query languages used for the development of KGs (e.g., based on the Neo4j technology). In this context, this paper provides a robust and efficient framework to systematically assess the efficiency of Large Language Models (LLMs) to support Text-to-Cypher conversion, focusing on the evaluation of open-source LLMs. The framework utilizes metrics and validators offered by an open-source software library that we developed, called CyVer. This study also assesses the impact of different schema representations of the KG on schema-aware query generation and the performance of LLMs on questions of different complexity requiring a depth of reasoning on the KG. A case study is described based on the application of the detailed framework in a KG with a large and complex schema that hosts data to track information related to the Sustainable Development Goals (SDGs). The experimental results demonstrate the effectiveness of the proposed framework, highlight the importance of the size of open-source models in the semantic comprehension of questions and the generation of valid Cypher queries, and stress the challenge for the generation of accurate queries in the case of questions requiring complex Cypher logic.

INDEX TERMS Cypher language, cypher query generation, knowledge graphs, large language model, Neo4j, question answering, text-to-cypher, validators.

I. INTRODUCTION

Large Language Models (LLMs) have gained prominence in Natural Language Processing tasks [1], [2], [3], [4] due to their excellence in grasping nuances in context and meaning of the human language. Their rapid growth, fueled by models like ChatGPT [5], has expanded their application in various tasks. The rise of LLMs has significantly advanced Question Answering (QA) systems over data stored in databases of different type (e.g., relational, non-relational). These QA systems take advantage of the remarkable capabilities of

the LLMs in understanding, reasoning, and generating text by following instructions. In parallel, the introduction of generative Artificial Intelligence (AI) techniques in such systems has opened new avenues for their adoption by users without a technical background.

The integration of LLMs with graph database models like Knowledge Graphs (KGs) offers a flexible decision-making QA system that supports faster data retrieval, adaptability, scalability, and reduced manual effort [6], [7]. KGs offer a sophisticated model structure that represents data as nodes interconnected with relationships. Unlike traditional relational databases, the graph model captures complex and semantic relationships between entities. Among graph

The associate editor coordinating the review of this manuscript and approving it for publication was Chang Choi^{ID}.

models, Labeled Property Graphs (LPGs) (e.g., those implemented based on Neo4j technology) are gaining interest due to their flexibility and ability to handle heterogeneous, structured, or unstructured data, with rich contextual metadata [8], while offering high performance over large datasets.

Recently, the field of LLMs that access and manage graph-structured data for QA systems has gained significant attention [9], [10]. However, it continues to pose considerable challenges. To query structured data in LPGs, the Cypher query language is adopted, which is designed for pattern-based retrieval. The Text-to-Cypher task aims to translate natural language questions into Cypher queries to retrieve data from KGs by leveraging the knowledge and capabilities offered by the LLMs. Therefore, the aforementioned task constitutes a crucial step in QA systems over KGs. This task holds great potential for non-expert users that want to interact with the KG without a need for knowledge of Cypher syntax. However, Text-to-Cypher techniques remain underexplored compared to their Text-to-SQL [11], [12] and Text-to-SPARQL [13], [14] counterparts, which are applied in relational databases and KGs described with other description languages such as the Resource Description Framework (RDF), respectively.

Employing an LLM in the Text-to-Cypher task adds an additional level of complexity, as the structure of Cypher queries is highly dependent on the underlying KG schema, which the LLM has not encountered during its training stage and therefore needs to be introduced through the prompt template. Related work in this field focuses mainly on KGs with a simple and small schema [15], as a larger and more complex schema is particularly challenging for LLMs due to semantic difficulties and context-length constraints. Moreover, LLMs are typically trained on a larger volume of SQL related data sets due to their widespread use, whereas Cypher training data are more limited due to its narrower adoption [16].

Attempting Text-to-Cypher without fine-tuning may lead to LLM hallucinations [17], resulting in invalid queries. Most existing work relies on proprietary models [9], [15] (e.g., GPT-4, InstructGPT-3) to ensure validity and success in generating Cypher queries. However, reliance on these models raises concerns about resource availability, openness, and costs, and opens the field in exploring open-source alternatives to make this task more accessible.

In this study, we focus on evaluating open-source LLMs that are cost-effective and require less computational power for the Text-to-Cypher task by using prompt engineering techniques. This evaluation process is performed on a large complex schema of a KG that hosts data to track information related to the Sustainable Development Goals (SDGs), namely the SustainGraph [18]. We aim to explore the performance of LLMs with sizes ranging from 2 billion to 72 billion parameters, and compare their success to the state-of-the-art proprietary model, gpt4o. To facilitate this evaluation and systematically assess the performance of a

model in the Text-to-Cypher task, we propose a framework with a set of metrics and validators to assess the quality of generated queries and their outputs. The framework is called the Metric-Based LLM Evaluation framework. The validators are offered as an open-source Python library. Additionally, we assess the impact of KG schema representation on schema-aware query generation to understand how different schema formats influence the ability of the model to generate accurate Cypher queries. We introduce three KG schema representations for instruction prompting that can be adopted for any KG schema to improve the LLM performance in the Text-to-Cypher task. The source code is available in a Gitlab Repository [19].

Through the evaluation process, we showcase that open-source LLMs can achieve competitive performance in the Text-to-Cypher task in a complex KG schema, with Qwen2.5-Coder-32B-Instruct reaching 67.95% success rate compared to gpt4o's 69.93%. We also find that tailoring prompt instructions for zero-shot prompting based on review of patterns in the outputs of the model enhances the performance of the model in Cypher syntax validity up to 73.3%. Finally, the results show that the varying level of detail in the KG schema representations affects the performance of the LLM in generating valid schema-aware queries.

The remainder of this work is structured as follows. Section II provides an overview of related work in the field of converting natural language to query code with LLMs. In Section III, we thoroughly present the proposed Metric-Based LLM Evaluation framework, including the description of each of the evaluation metrics and validators. Section IV describes the experimental setup for the evaluation of a set of LLMs over a complex KG schema, including the structure of the dataset, the prompting techniques and the representations of the KG schema introduced. Section V presents the evaluation results and key findings of the review and error analysis of the outputs. Finally, Section VI concludes the paper and outlines limitations and future research ideas.

II. BACKGROUND INFORMATION ON QUESTIONS ANSWERING MECHANISMS

In the following, we provide information on the adoption of LLMs for the development of QA systems over databases. We focus on the current status of work for relational and graph databases. For the latter, we examine databases where their schema is based on RDF or developed in the form of an LPG.

A. RELATIONAL DATABASES

Text-to-SQL QA systems are among the most extensively studied topics in the field of natural language interfaces for databases. With the advent of LLMs, there has been remarkable progress in this domain. Existing work spans from large benchmark datasets to prompt engineering, fine-tuning, or hybrid techniques covering various aspects in the Text-to-SQL task.

Spider [20], [21], with three robustness variants, namely Spider-DK [22], Spider-Syn [23], and Spider-Realistic [24], WikiSQL [25], KaggleDBQA [26] and COSQL [27] are the most widely used datasets for Text-to-SQL QA systems. BIRD [28] is also a pioneering cross-domain dataset that evaluates the impact of large database contents on Text-to-SQL parsing. Existing work [21], [29], [30], [31], [32] had led to impressive performance over the aforementioned benchmark datasets.

Prompt engineering techniques or adaptive strategies in Text-To-SQL tasks [10], [29], [31], [32], [33] have been widely explored. Various frameworks are detailed for the Text-To-SQL task, such as a framework that decouples schema linking and skeleton parsing [21], a divide-and-conquer framework with structured SQL generation [29], and a hybrid prompt strategy or solutions optimizing user-written prompts for better LLM performance [30]. Most of these studies evaluate closed-source models, such as GPT variants, which have shown superior performance. Existing studies on the Text-to-SQL task solve challenges that arise from a large number of schema items or complex SQL operators [21], [31]. These challenges are further compounded by poor generalization and recommended representation prompts, incorporation of additional parameters on open-source models. Fine-tuning techniques [30] and error analysis are also developed to improve performance and reveal the main sources of inaccuracies [28], [31], [32].

B. GRAPH DATABASES

Due to the increasing interest in graph databases based on semantic RDF and LPG models, the tasks of Text-to-SPARQL and Text-to-Cypher translation have received significant attention.

1) RESOURCE DESCRIPTION FRAMEWORK DATABASES

For the Text-to-SPARQL task, both rule-based [34] and machine learning-based approaches [35] have been developed to map natural language to SPARQL queries, resulting in benchmark datasets for evaluation and training. The LC-QuAD 1.0 [36] dataset was one of the first, with 5,000 pairs of natural language questions to SPARQL, although with rather simple and less complex queries. This was followed by the LC-QuAD 2.0 [37] dataset, which increased both the scale and complexity, containing 30,000 pairs compatible with Wikidata and DBpedia 2018 KGs. The DBpedia Neural Question Answering (DBNQA) dataset [38] further expanded the scale, offering 894,499 question-query pairs.

To achieve higher complexity in questions and queries and a larger vocabulary size, the Spider4SPARQL benchmark dataset was introduced featuring almost 10,000 manually generated natural language questions and 4,721 complex SPARQL queries based on various KGs in different domains [39]. Other datasets focus on semantically rich repositories like Wikidata, exemplified by

Instruct-to-SPARQL [40], a curated dataset designed for fine-tuning models.

More general approaches propose the adoption of neural network models [14], [41] to encode natural language to SPARQL query language, mitigating the labor-intensive process of adapting systems to new domains. Similarly, [42] introduced a universal QA system that converts a question into an intermediate abstract representation using a neural sequence-to-sequence model, which is then mapped to a SPARQL query based on the structure of the target KG.

2) LABELED PROPERTY GRAPH DATABASES

Several efforts have been made in generating high-quality semantic parsing datasets from natural language to Cypher query language. The SpCQL dataset [43] is an early contribution, providing 10,000 manually annotated natural language questions paired with their Cypher queries based on the OwnThink Neo4j KG. However, its focus on the Chinese language limits its broader applicability. The shortage of large-scale public datasets was emphasized in [44], where they took advantage of existing Text-to-SQL datasets and converted the SQL statements to Cypher statements to generate a Text-to-Cypher corpus. More recent approaches, have focused also on alleviating the labor-intensive nature of data annotation, and introduced the SyntheT2C methodology [45] for constructing synthetic Query-Cypher pairs. The MedT2C dataset was generated through this methodology, based on two medical KG databases. Further datasets based on open-source graph databases have been developed [17], [46]. In the latter, they combined instances from publicly available sources, constructing the Neo4j-Text2Cypher dataset, with over 44,000 samples. These approaches attempt to offer a well-structured collection for training and testing models in the Text-to-Cypher task.

Benchmarking and fine-tuning models on the Text-to-Cypher task have been recently explored. The Neo4j-Text2Cypher dataset [17] was leveraged to fine-tune the open-source LLMs LLama3.1-8B-it and gemma2-9B-it, which demonstrated improved performance, but were still outperformed by closed models such as OpenAI's GPT and Google's Gemini models. GPT4Graph [9] introduced a benchmarking framework to evaluate LLMs on graph-structured data, including zero-shot and few-shot techniques, with results showing that input design significantly impacts performance. The Text-to-Cypher task was evaluated on the MetaQA dataset using different versions of the InstructGPT-3 LLM.

These evaluations have underscored the challenges of LLMs in converting natural language to Cypher. In response to these difficulties, several approaches have been proposed to enhance the accuracy and reliability of Text-to-Cypher systems. In particular, the R3-NL2GQL approach [46] proposes the use of small fine-tuned and large foundation models to perform ranking, rewriting and refinement tasks. A similar approach was introduced in [47], where the NAT-NL2GQL

multi-agent framework consists of 3 specialized agents for pre-processing, generating Cypher queries and refining them. Both approaches reflect a growing emphasis on modular and cooperative systems to address the complexities inherent in LLM-driven query generation.

C. CROSS-TECHNOLOGY APPROACHES

Recent work has explored cross-technology approaches to enhance LLM-driven query generation over diverse data models. In [48] property graph views over large RDF graphs, such as Wikidata, have been proposed as a more LLM-friendly alternative for querying using Text-to-Cypher instead of Text-to-SPARQL. To support this approach, an RDF-to-property graph conversion engine was developed, and CypherBench was introduced as a benchmark consisting of 11 large-scale, multi-domain property graphs. Extending beyond single query language approaches, the SM3-Text-to-Query benchmark [16] was recently introduced, mapping 10,000 diverse natural language questions for healthcare datasets to their query in four different languages, namely SQL, MongoDBQL, Cypher, and SPARQL. This benchmark allows evaluating text-to-query models across relational, document, and graph databases. Evidently, the evaluated LLMs (GPT-3.5 and Llama3-70b, Llama3-8b) exhibit mostly consistent performance patterns across query languages, but their performance varies, with SQL outperforming SPARQL and Cypher.

D. MOTIVATION

Following existing work in the field, we provide some information about our main motivation for the work in the current manuscript. With the widespread adoption of LPGs, there has been a growing interest in the Text-to-Cypher task. However, this task remains underexplored, particularly due to the absence of mature open-source frameworks for evaluating the performance of LLMs in this context, especially when dealing with complex KG schemas and open-source models. Our methodology aims to address this gap by enabling the systematic assessment of LLM generated Cypher queries over large and structurally diverse KGs. We also introduce dedicated metrics and validation mechanisms to ensure the reliability of generated queries and the comparability of LLM performance.

III. METRIC-BASED LLM EVALUATION FRAMEWORK

The Metric-Based LLM Evaluation framework is designed to systematically assess and refine the performance of LLMs in the Text-to-Cypher task through structured evaluation criteria. By offering a standardized methodology, the framework ensures adaptability across different datasets and applications. It enables rigorous evaluation of the performance of LLMs in generating Cypher queries from natural language input, providing compatibility with any KG schema. It presents an approach that integrates prompt engineering techniques with a comprehensive evaluation stage, facilitating the identification of the most reliable

and trustworthy LLM for use in QA systems, Retrieval-Augmented Generation (RAG) applications, agentic AI, and other knowledge-driven AI applications. The Metric-Based LLM Evaluation framework consists of three main stages; the Prompt Engineering stage, the Evaluation Metrics stage, and the Review and Error Analysis stage. An overview of the proposed framework is provided in Fig. 1.

In the Prompt Engineering stage, instruction-based prompting is designed to guide the LLM in generating Cypher queries that adhere to a given KG schema. A template-based approach is used, where natural language questions from a dataset are integrated into a prompt template. This template provides the necessary context, including system instructions for the model, as well as KG schema description of node labels and relationship types with their properties and one-hop paths. This type of prompt aims to instruct the LLM to generate Cypher queries that are both syntactically valid and semantically meaningful. Few-shot or zero-shot prompting techniques may also be employed depending on the complexity of the task, the KG schema, the parameter space of the model, or the preferences of the user. The process of crafting these instructions and identifying the best way to describe the KG schema involves an iterative refinement, as different prompt formulations may lead to variations in the accuracy of the LLM. The effectiveness of a prompt is subject to user judgment and empirical testing, which may require multiple iterations to optimize the LLM performance.

The Evaluation Metrics stage is responsible for the examination of the quality of the generated queries. To evaluate a generated Cypher query we consider various aspects including its syntax correctness, structural and logical consistency, similarity to the ground-truth Cypher query, and similarity of the retrieved output. This stage employs a multi-level evaluation process. In the beginning of the stage, the generated queries are evaluated through the Syntax Validator, which ensures that the query is correctly structured and syntactically valid according to the Cypher formal grammar. Syntactically valid queries are further evaluated through three categories of metrics and validators. The first category includes Cypher validators, namely the Schema and Properties Validators. The Schema Validator ensures that the generated query conforms to the expected KG schema, i.e. the alignment of the nodes, relationships, and one-hop paths with the structure of the schema. Then, the Properties Validator ensures that the generated query accesses only valid properties for nodes and relationships based on their labels or types. The second category of metrics, called Cypher Metrics, assesses the closeness of the generated syntactically correct query to the ground-truth query. These metrics include the Rouge-L, Jaccard similarity, BLEU, and Jaro-Winkler similarity scores. For validated queries in syntax, schema, and properties, in the third step, an output-based evaluation is conducted by executing them on the KG database and comparing the retrieved results against the expected output of the ground-truth query. The pass@1 and the Jaccard Output similarity scores determine whether

Metric-Based LLM Evaluation Framework

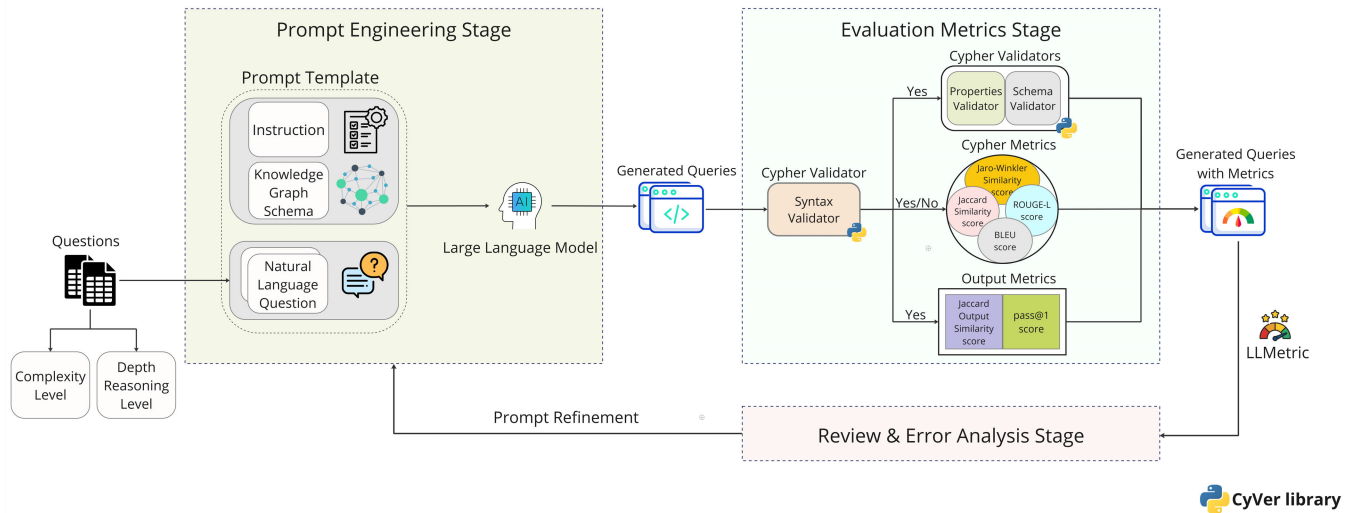


FIGURE 1. The metric-based LLM evaluation framework.

the query retrieves exactly correct or almost correct results, respectively, to validate the semantic validity of the result. If a Cypher query is found to be syntactically incorrect, then it is only examined through the Cypher Metrics, as it may contain additional text, such as an explanation from the LLM, or non-query text and therefore cannot be executed on the KG database. The three types of evaluation metrics are explained in detail in Sections III-A, III-B and III-C.

Through these metrics, we can comprehensively evaluate the performance of LLMs on the Text-to-Cypher task. To provide a holistic evaluation, we introduce a composite metric, termed LLMetric, detailed in Section III-D, which aggregates the aforementioned evaluation criteria into a single score. This metric enables direct comparison of multiple LLMs on a given dataset, offering a standardized benchmark for assessing the performance of a model in Cypher query generation.

The Review and Error Analysis stage involves a human-in-the-loop process to examine the LLM outputs and the errors detected during the Evaluation Metrics stage. This process can be further enhanced by leveraging an auxiliary LLM to autonomously provide feedback, detect patterns, and suggest refinements. The insights of this stage are used to refine the prompts, leading to iterative improvements in query generation to enhance the performance of the LLM.

A. CYPHER VALIDATORS

In this section, we present *CyVer*, an open-source Python library designed to offer the key validators in the Evaluation Metrics stage of the framework. *CyVer* is made publicly available in a Gitlab repository [49], ensuring accessibility and reproducibility. The library consists of three main classes that examine a Cypher query concerning its correctness in

TABLE 1. Overview of *CyVer* classes.

Class	Method	Description
SyntaxValidator: Validates the Cypher queries for correct syntax and label consistency	validate	It examines if the Cypher query has conflicting node labels or relationship types. It returns a boolean variable, indicating if the Cypher query is syntactically correct or not.
SchemaValidator: Ensures that the Cypher queries align with the predefined schema structure	extract	It returns the node labels, the relationship types and the one-hop paths accessed in the provided Cypher query.
	validate	It returns a score in the range [0,1] that is a weighted average of the correct nodes, relationships and one-hop paths over the accessed nodes, relationships and one-hop paths.
PropertiesValidator: Examines the correctness of property access in the Cypher queries against the predefined schema structure	extract	It returns two dictionaries. The first returns the variables in the Cypher query with the accessed properties. The second returns the node s or relationship types, both mentioned in the query and inferred by variables, with their properties.
	validate	It returns a score in the range [0,1], that is the number of correct accessed properties over the total number of accessed properties. It returns None, if no properties are accessed.

terms of Cypher syntax, as well as schema, and properties access based on the given KG schema. These classes are detailed in Table 1.

The *CyVer* library supports both simple and complex Cypher queries, including those without explicitly defined labels, such as `MATCH (n) RETURN n`. For this type of queries, it employs the inference function, which is necessary for Schema and Properties Validators, since it infers the label of nodes or types of relationships based on the provided schema. In this way, it is ensured that the validators return a valid evaluation score.

B. CYPHER METRICS

The Cypher Metrics are commonly used to evaluate the quality of text or string matching. In our case, these metrics are useful to examine the similarity between the generated and ground-truth Cypher query. The Jaro-Winkler character-based similarity score measures how different the Cypher queries are and how much the generated one needs to be modified to be the same as the reference Cypher query. While Jaro-Winkler similarity score is designed for comparing strings with small differences, Jaccard similarity score is more suitable for comparing sets of words between the two Cypher queries. Jaro-Winkler is useful for comparing node/relationship properties or labels/types with minor differences, while Jaccard compares a set of common variables, clauses, and other Cypher keywords used in the queries.

Focusing on n-gram (a sequence of n adjacent symbols in particular order) matches or the structure of the generated Cypher query, ROUGE-L precision, recall, F1-score and BLEU score are selected for the Evaluation Metrics stage. ROUGE-L and BLEU scores measure the overlap of the longest common subsequence and n-gram between the generated and the reference query respectively.

C. OUTPUT METRICS

The Output Metrics facilitate the comparison between the retrieved output of the generated Cypher query with the corresponding one of the ground-truth query. Two metrics assess the evaluation of the retrieved output. The pass@1 score returns True if the generated query retrieves the same results as the ground-truth Cypher query. A less stringent metric, the Jaccard Output similarity score, measures the Jaccard similarity between the responses produced by the generated query and the ground-truth Cypher query. This metric is particularly useful in cases where the output is nearly correct but not identical.

D. LLMETRIC

In this study, we propose a composite metric to evaluate the performance of LLMs in the Text-to-Cypher task. This metric is based on the following formula:

$$\begin{aligned} \text{LLMetric} = & w_1 \cdot \text{Pass@1 Rate} \\ & + w_2 \cdot \text{KG Valid Query Rate} \\ & + w_3 \cdot \text{Jaccard Factor} \\ & + w_4 \cdot \text{JaRou Factor} \end{aligned}$$

where w_1, w_2, w_3, w_4 are weighting factors.

The two main components of the LLMetric are the Pass@1 Rate and the KG Valid Query Rate. The KG Valid Query Rate is defined as the ratio of generated Cypher queries that are syntactically correct (i.e., Syntax Validator=True), respect the schema (i.e., Schema Validator=1) and the properties (i.e., Properties Validator=1 or None) of the given KG, to the total number of Cypher queries generated. This component is essential, as it reflects the reliability of the LLMs in producing accurate Cypher queries without errors or misinterpretation

for efficient retrieval of data. Additionally, the correctness of the generated outputs is equally important and is measured by the Pass@1 Rate. The Pass@1 Rate, also known as exact execution accuracy [20], measures the percentage of Cypher queries generated that return identical results to the ground-truth query (i.e., pass@1=True). To accommodate a wider range of acceptable queries, two adjustment factors are included in the LLMetric formula, the Jaccard Factor and the JaRou Factor. These factors allow for the inclusion of queries that respect the schema and properties of the given KG but do not necessarily produce the same output.

The Jaccard Factor is an adjustment to the Pass@1 Rate that measures the ratio of KG Valid generated queries with outputs that maintain similarity to the ground-truth but are not identical (i.e., pass@1=False and Jaccard Output similarity score > 0) relative to the KG Valid generated queries with incorrect outputs (i.e., pass@1=False).

To further favor models that not only generate KG Valid queries, but also generate queries with a high degree of similarity to the ground-truth Cypher queries and not identical output, the LLMetric is augmented by the addition of the JaRou Factor. To measure the similarity between the KG Valid generated queries and the ground-truth Cypher queries, we define the JaRou as the average score of ROUGE-L, F1-score and Jaro-Winkler similarity score. Higher JaRou values indicate that the generated Cypher query can match exactly the ground-truth one with small adjustments. The JaRou Factor is defined as the ratio of KG Valid generated queries that while not producing the desired output (i.e., pass@1=False), exhibit a high degree of similarity to the ground-truth query (JaRou $\geq$ 0.8), to the total number of KG Valid generated queries that retrieved incorrect outputs. To allow for a wider range of similarities, the threshold of JaRou $\geq$ 0.8 could be adjusted or relaxed.

In our case, we set $w_1 = 0.3$, $w_2 = 0.4$, $w_3 = 0.2$ and $w_4 = 0.1$. These weights are assigned to equally balance the importance of two types of evaluations: the one regarding the correctness of the Cypher query (KG Valid Query Rate and JaRou Factor), and the other focusing on the correctness of the retrieved output (Pass@1 Rate and Jaccard Factor). The KG Valid Query Metric is given a higher weight because it ensures that the generated query adheres to the essential requirements of schema, property, and syntax correctness. Meanwhile, the Pass@1 Rate is more strict and might fail to account for the correctness of the answers if the retrieved output has a different structure. To balance this and allow for some flexibility, the weights of the Jaccard Factor and the Pass@1 Rate have close values.

IV. KNOWLEDGE GRAPH AND EXPERIMENTAL SETUP

A. THE SUSTAINGRAPH

The Text-to-Cypher task is performed using the SustainGraph [18], which is a KG developed as an LPG using the Neo4j technology. Using the SustainGraph for the Text-to-Cypher task is challenging due to its complex schema.

TABLE 2. Detailed explanation of the Complexity sublevels.

Complexity Level	Query Structure	Description	Indicative clauses, Functions and Operators
Easy	Basic clauses	Simple filtering conditions with optional WHERE clause	MATCH, OPTIONAL MATCH, WHERE, RETURN
Medium	All operators and basic clauses	Basic sorting, limiting results, basic aggregation, all operators	ORDER BY, LIMIT, COUNT, DISTINCT, EXISTS, IN, AND, OR, MIN, MAX, AVG
Hard	Advanced clauses	Using COLLECT to aggregate data, FILTER to apply conditions on collections, and WITH to chain subqueries or perform intermediate operations, along with advanced constructs like REDUCE, FOREACH, and conditional logic with CASE	FILTER, COLLECT, UNWIND, WITH, FOREACH, REDUCE, CASE

labels and relationship types of the SustainGraph schema. The following aspects are considered through the dataset development process: Cypher pattern coverage, Cypher consistency, Question clarity, and Schema alignment [20].

The Cypher pattern coverage aims to ensure that the dataset includes queries representing a comprehensive range of Cypher patterns and clauses. The Cypher consistency examines alternative Cypher query formulations for a given question that demonstrate different ways to structure the queries while retrieving the same result. These alternative formulations involve variations in filtering based on properties, either by incorporating them inside the MATCH pattern or by using the WHERE clause, as well as differences between using a single MATCH clause versus multiple separate patterns. This ensures the robustness of the evaluation process that accommodates various potential outputs from the LLM. Question clarity ensures that the questions follow a more precise and less ambiguous approach, by asking information that could be answered by the given database and by stating exactly the relationship types or node labels according to the schema. To ensure alignment of the KG schema in the generated query, the evaluation dataset includes questions that explicitly specify the relevant paths in the KG schema.

The questions are designed based on two categories; depth reasoning and complexity level. The depth reasoning level concerns the composition of Cypher queries based on the required depth of data interaction, following the approach proposed in [54]. More specifically, we have adopted the first two levels of the depth reasoning categorization to construct our evaluation dataset. The first level suggests explicit facts (i.e., queries without requiring any additional reasoning). In our case, explicit facts are translated into Cypher queries regarding only nodes. The second adopted

TABLE 3. Sample data from the Text-to-Cypher SustainGraph Dataset.

Depth Reasoning Level	Sublevel	Complexity Level	Question	Ground-truth Cypher Query
Explicit Facts	Nodes	Easy	What is the title of the goal with code 2?	<pre>MATCH (n:Goal {code : '2' }) RETURN n.title</pre>
Explicit Facts	Nodes	Medium	Which are the types of European Green Deal (EGD) documents?	<pre>MATCH (n:EGD) RETURN DISTINCT n.type</pre>
Explicit Facts	Nodes	Hard	What are the policy areas of the European Green Deal (EGD) documents?	<pre>MATCH (n:EGD) UNWIND n.policyArea AS pa RETURN DISTINCT pa AS policyAreas</pre>
Implicit Facts	One-hop	Easy	What are the target codes that goal with code 8 has?	<pre>MATCH (n:Goal{code:'8'}) -[:HAS_TARGET]-> (t:Target) RETURN t.code</pre>
Implicit Facts	One-hop	Medium	Which is the average weight of all the EGD documents associated with goal with code 12?	<pre>MATCH (n:EGD)-[a:ASSOCIATED_WITH]-> (g:Goal {code:'12'}) RETURN AVG (a.weight)</pre>
Implicit Facts	One-hop	Hard	Does the indicator with code sdg_06_60 has observations in area with name Greece?	<pre>MATCH (:Indicator{code:'sdg_06_60'})-[:HAS_OBSERVATIONS]-> (:Area{name:'Greece'}) RETURN CASE WHEN COUNT(r)>0 THEN TRUE ELSE FALSE END AS hasObservations</pre>
Implicit Facts	Multi-Hop	Hard	For goal that has targets which have indicators, what are the top 3 goals with the highest number of indicators coming from the source with name UN_SDG?	<pre>MATCH (g:Goal)-[:HAS_TARGET]-> (t:Target) -[:HAS_INDICATOR]-> (i:Indicator) -[:COMES_FROM]-> (so:Source {name:'UN_SDG'}) WITH g,COUNT (i) AS inds RETURN g.code, g.title, inds ORDER BY inds DESC LIMIT 3</pre>

level regards implicit facts (i.e., the queries which require a certain amount of common sense reasoning or fundamental logical conclusions and are not immediately apparent). In the context of a KG, this level is subdivided into two types of queries; those involving one-hop paths and those involving multi-hop paths. The complexity level classifies the questions into three categories; easy, medium, and hard, based on the complexity of the Cypher queries as illustrated in Table 2. An indicative set of questions from the dataset is presented in Table 3.

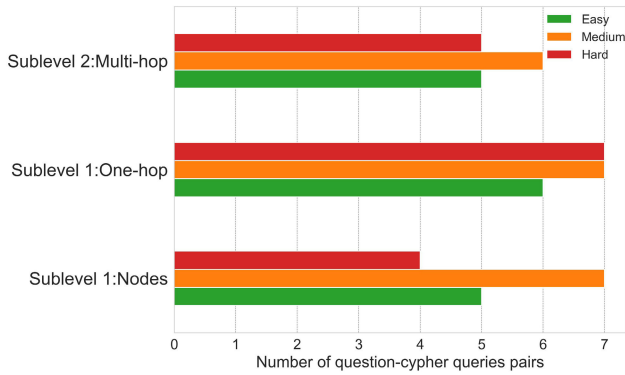


FIGURE 4. The distribution of question-Cypher query pairs across the complexity sublevels for each depth reasoning sublevel.

Fig. 4 illustrates that the 52 pairs of questions and ground-truth Cypher queries are distributed fairly evenly across the depth of reasoning levels. Each level is almost equally divided among the complexity levels. The evaluation dataset is openly accessible through a Gitlab repository [19].

C. KNOWLEDGE GRAPH SCHEMA REPRESENTATION

To ground the models in the SustainGraph through in-context learning, we provide a structured description of the KG schema as input to the prompt. This schema outlines the node labels and relationship types, along with their associated property names and types, and the one-hop paths of the SustainGraph. The Langchain framework [55] proposes a KG schema description that includes all node labels and relationship types. However, in our approach, we refine this schema by removing generic labels from multi-labeled nodes (i.e., in our case labels GeoArea and Typology). This reduction aims to decrease the size of the overall schema description, without any information loss, and enhance the performance of the LLM by minimizing ambiguity and reducing complexity. To examine the robustness of the schema description in the prompt and whether the description style affects the model's successful output, we proposed three different ways of describing KG schemas. Each schema description type is presented in Table 4 and indicative detailed representations of the SustainGraph schema are provided in the Appendix in Table 15.

D. OVERVIEW OF SELECTED LARGE LANGUAGE MODELS

Thirteen text-to-text decoder-only LLMs are selected to be evaluated. Focus is given on open-source state-of-the-art LLMs with fewer than 100 billion parameters, as these models are more accessible for research and practical deployment with reduced computational costs. Furthermore, the gpt4o model is examined to provide a performance benchmark against a state-of-the-art proprietary model.

To systematically assess these models, we classify them into three categories based on their size: small, medium, and large. Table 5 summarizes the selected models per category. For the Text-to-Cypher task the instruction-tuned version of

TABLE 4. The different types of KG schema description.

KG Schema Description Type	Description
Full schema	Full node definitions with labels, property names and types, as well as one-hop paths with full definition of both nodes and relationships with labels and types, property names and types.
Nodes and Paths	Full node definitions with labels, property names and types, as well as one-hop paths with full definition of relationships but avoiding redundant property details for nodes that are already explicitly defined.
Only Paths	One-hop paths with full definition of both nodes and relationships with labels and types, property names and types.
Langchain schema	Full node definitions with labels, property names and types, full relationship definitions with types, property names and types, as well as one hop-path paths without avoiding redundant property details for nodes and relationships that are already explicitly defined.

TABLE 5. The model size and category of the selected LLMs.

Category	Model	Model size (parameters)
Small	google/gemma-2-2b-it	2.61 Billion
	Qwen/Qwen2.5-3B-Instruct	3.09 Billion
	Qwen/Qwen2.5-Coder-3B-Instruct	3.09 Billion
	meta-llama /Llama-3.2-3B-Instruct	3.21 Billion
Medium	mistralai/Mistral-7B-Instruct-v0.3	7.25 Billion
	Qwen/Qwen2.5-7B-Instruct	7.62 Billion
	Qwen/Qwen2.5-Coder-7B-Instruct	7.62 Billion
	meta-llama /Llama-3.1-8B-Instruct	8.03 Billion
	google/codegemma-7b-it	8.54 Billion
Large	Qwen/Qwen2.5-Coder-32B-Instruct	32.8 Billion
	mistralai/Mixtral-8x7B-v0.1	46.7 Billion
	Qwen/Qwen2.5-72B-Instruct	72.7 Billion
	OpenAI/gpt-4o-2024-08-06	

the models is selected, where available, as these models are optimized to better align with natural language instructions. Additionally, code-specialized versions of certain models are included, which are trained in structured programming languages to assess their effectiveness in schema-guided Cypher query generation.

We evaluate the proprietary model through its official Application Programming Interface (API), open-source medium-sized and large-sized models using Hugging Face Inference API and small-sized lightweight open-source models in a server with NVIDIA Tesla T4 16GB GPU. To enable the evaluation of certain medium-sized models on the GPU, 4-bit and 8-bit quantization is applied. Through quantization, lower usage of computational resources and higher inference speed are achieved while preserving a reasonable level of performance of the LLM [56]. However, LLMs with over 30 billion parameters remain too resource intensive to run them locally on the GPU even with quantization techniques.

In the Text-to-Cypher task, we examine two decoding strategies to achieve more deterministic Cypher outputs based on the natural language questions. The first refers to greedy search, by selecting the next token with the highest probability as the next token in the sequence. The

TABLE 6. First version of instructions in prompt template for Cypher query generation.

Prompt instructions
You are an AI assistant designed to convert natural language question to cypher. Based on the Neo4j graph schema : {schema} write a Cypher query. Return only Cypher query, no backticks, no explanations nothing else.

other regards top-k sampling of the next token, but with low temperature values to favor high-probability tokens. Specifically, for random sampling we set temperature to 0.01, top_k parameter to 20 and top_p to 0.9, to mimic the greedy behavior but allow for a certain degree of randomness. The maximum number for generated tokens (max_new_tokens) was set to 512 for all models, where applicable, ensuring a fair and consistent comparison of the generated outputs.

V. EXPERIMENTAL RESULTS

In this section, we present the insights derived from the application of the proposed framework. In Section V-A, we apply the proposed framework, by defining and evaluating instruction prompts for each LLM based on the standardized prompt template. Following this, the framework is employed in Section V-B to evaluate the effect of the proposed KG schema representations and in Section V-C the overall performance of the selected LLMs, with a thorough review of their failures on the basis of the difficulty and complexity of the questions. In Section V-D, the framework is applied to explore the best decoding strategies for each LLM and determine their optimal configuration. Additionally, the effect of quantization on performance is examined for several medium-sized models. In Section V-E a comprehensive validator-based performance analysis and classification of incorrect generated Cypher queries are performed. Finally, in Section V-F prompt engineering techniques are employed to refine the prompt of the model with the highest performance score.

A. GENERAL INSTRUCTION PROMPT DESIGN

At first, we evaluated all models in the Text-to-Cypher task using a general uniform zero-shot prompt, based on the prompt template, to ensure comparability between them. The models were configured with a sampling decoding strategy, as detailed in Section IV-D. The initial prompt instructs each model to generate only the Cypher query as plain text, based on the KG schema, given an input question. The detailed instruction prompt is provided in Table 6.

However, several models exhibited unexpected patterns in their generated outputs. Specifically, gemma-2-2b-it, Qwen2.5-7B-Instruct, Qwen2.5-Coder-7B-Instruct and codegemma-7b-it added Cypher code formatting in some outputs by enclosing the generated query within " `Cypher . . . `". Meanwhile, Mistral-7B-Instruct-v0.3 and Mixtral-8 × 7b-v0.1 enclosed certain Cypher queries in backticks and added natural language explanations. To address these

TABLE 7. Second version of instructions in prompt template for Cypher query generation.

Prompt instructions
You are an AI assistant designed to convert natural language question to cypher. Based on the Neo4j graph schema : {schema} write a Cypher query, as plain text without any code block formatting or backticks. Only write the query text like this: MATCH (n) RETURN n Return only Cypher query, no backticks, no explanations nothing else.

TABLE 8. Third version of instructions in prompt template for Cypher query generation.

Prompt instructions
You are an AI assistant designed to convert natural language question to cypher. Based on the Neo4j graph schema : {schema} write a Cypher query, as plain text without any code block formatting or backticks. Only write a cypher query that matches and returns nodes with explicitly defined labels and/or properties. For example: MATCH (n:LabelName) RETURN n. Do not return nodes without labels or properties. Queries like MATCH (g) RETURN g are not acceptable. Return only Cypher query, no explanations nothing else.

formatting issues encountered at the Review and Error Analysis stage of the proposed framework, a prompt refinement process was conducted. Specifically, we refined the instruction prompt for these models by instructing them specifically to return plain text without any code block formatting or backticks. The updated instruction prompt is available in Table 7.

Following the new prompt, gemma-2-2b-it generated some queries that were agnostic to the input question, meaning that they lacked specific labels, types or properties. For example, it produced generic queries like MATCH (n) RETURN n, which returned the entire graph without relevant filtering. The instruction prompt was further adjusted for this model, as described in Table 8, including a specific instruction to prevent the generation of such queries.

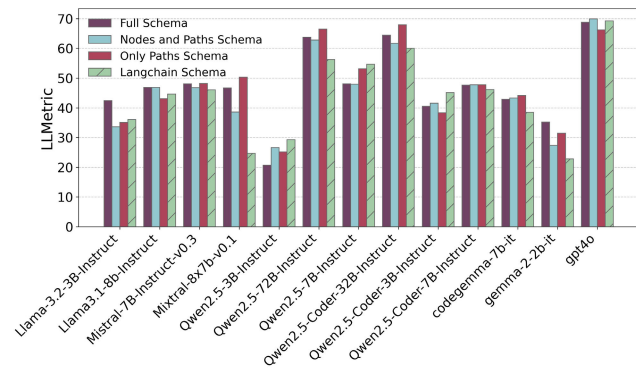
As shown in Table 9, these prompt adaptations led to significant improvements in the correctness of the Syntax Validator values of the queries generated in these models. In this way, with small variations for several models, we end up with a general instruction prompt to evaluate the models.

B. KNOWLEDGE GRAPH SCHEMA REPRESENTATION COMPARISON

Using the selected instruction prompts for each model, the framework was applied, as an ablation experiment, to investigate how the KG schema representation in the prompt impacts the performance of an LLM in the Text-to-Cypher task. Evaluating the models with the sampling decoding strategy configuration, we present the performance of each model comparing our three proposed KG schema representations against LangChain's default schema in Fig. 5.

TABLE 9. Performance of LLMs before and after adapting prompts, based on the percentage of Cypher Correct queries.

Model	Schema Type	Before	After	Improvement
Qwen2.5-7B-Instruct	Full Schema	44.2 %	75 %	30.8%
	Node and Paths Schema	46.2 %	80.8 %	34.6%
	Only Paths Schema	32.7%	75 %	42.3%
Qwen2.5-Coder-7B-Instruct	Full Schema	7.7 %	75 %	67.3%
	Node and Paths Schema	7.7%	80.8%	73.1%
	Only Paths Schema	3.8%	75 %	71.2%
codegemma-7b-it	Full Schema	65.4 %	80.8 %	15.4%
	Node and Paths Schema	63.5 %	71.2 %	7.7%
	Only Paths Schema	67.3 %	84.6%	17.3%
Mistral-7B-Instruct-v0.3	Full Schema	15.4 %	80.8 %	65.4%
	Node and Paths Schema	3.8 %	71.2 %	67.4%
	Only Paths Schema	11.5 %	75 %	63.5%
Mixtral-8x7B-v0.1	Full Schema	38.5 %	61.5 %	23%
	Node and Paths Schema	42.3 %	63.5%	21.2%
	Only Paths Schema	51.9%	78.8 %	26.9%
gemma-2-2b-it	Full Schema	23 %	80.8 %	57.8%
	Node and Paths Schema	23 %	82.7 %	59.7%
	Only Paths Schema	15.4 %	78.8 %	63.4%

**FIGURE 5.** Comparison of the LLMetric across the different KG schema representations in the prompt templates.

The findings highlight two key observations. First, the structure of the KG schema influences the performance of the LLM; however, no single schema representation consistently yields the best results across all models. Second, in 10 out of the 13 LLMs examined, our proposed schema representations outperformed the LangChain schema. Considering their improved performance and the added benefit of a reduced prompt context length, we focus on these three schema options, named Full Schema, Only Paths Schema, and Nodes and Paths Schema, for further evaluation and comparison in the following sections.

C. COMPREHENSIVE PERFORMANCE REVIEW

1) EVALUATION METRICS OVERVIEW

In the Review and Error Analysis stage, we examine the performance results of the selected open-source LLMs and gpt4o for the three suggested KG schema representations in the selected instruction prompts. Fig. 6 provides an overview of the performance results regarding the KG Valid Query Rate, Pass@1 Rate and Cypher Syntax Correct Rate across the different LLMs for the different KG schema representations.

a: PASS@1 RATE

It highlights that only open-source Qwen models with more than 32 billion parameters, as well as gpt4o successfully answered more than 50% of the questions with the expected correct output result (pass@1=True). Although Mixtral-8 × 7b-v0.1 accesses a vast parameter space, around 47 billion parameters, it has not been able to answer more than 50% questions with the expected output result and its performance is comparable to that of models with 7-8 billion parameters. Regarding medium-sized models, Qwen2.5-7B-Instruct, Qwen2.5-Coder-7B-Instruct and Mistral-7B-Instruct-v0.3 succeeded in correctly answering more than 30% of the questions. It is impressive that Qwen2.5-Coder-3B-Instruct, a model with a small space of parameters, answered more questions correctly across all schema representations compared to the medium-sized model codegemma-7b-it.

b: CYPHER SYNTAX CORRECT RATE

Regarding the Cypher Syntax correctness, only gpt4o achieved a Cypher syntax success rate greater than 90%. In addition, Qwen models with more than 32 billion parameters also produce more than 85% queries with correct syntax. It is remarkable that some LLMs with a smaller parameter space (Llama-3.2-3B-Instruct and codegemma-7b-it) answered more than 85% questions with a correct Cypher result. Ultimately, all models with at least one KG schema representation answered more than 50% questions with syntactically correct Cypher queries.

c: KG VALID QUERY RATE

In the Text-to-Cypher task, it is vital to choose an LLM that accurately respects the node, the relationships and the one-hop paths as well as the properties outlined in the KG schema representation within the prompt template. For this reason, the KG Valid Query Rate showcases how many produced Cypher queries satisfy syntax, schema, and property correctness. Once again, the Qwen 32-72B models as well as gpt4o produced the most KG Valid Cypher queries. Among the large-sized open-source models, only Qwen2.5-72B-Instruct generated more than 85% KG Valid queries, while Qwen2.5-Coder-32B-Instruct achieved over 75% but less than or equal to 80%. Mistral-7B-Instruct-v0.3, Qwen2.5-7B-Instruct and Qwen2.5-Coder-7B-Instruct models generated over 60% but less than or equal to 70% KG Valid queries. Across most of the different schema representations, all medium-sized models generated more than 50% KG Valid queries, while all small-sized models were unable to attain this level of accuracy.

d: LLMETRIC

The LLMs were evaluated on various metrics, as well as composite metrics such as the LLMetric presented in Section III-D. This evaluation provides a comprehensive understanding of their performance and is depicted in

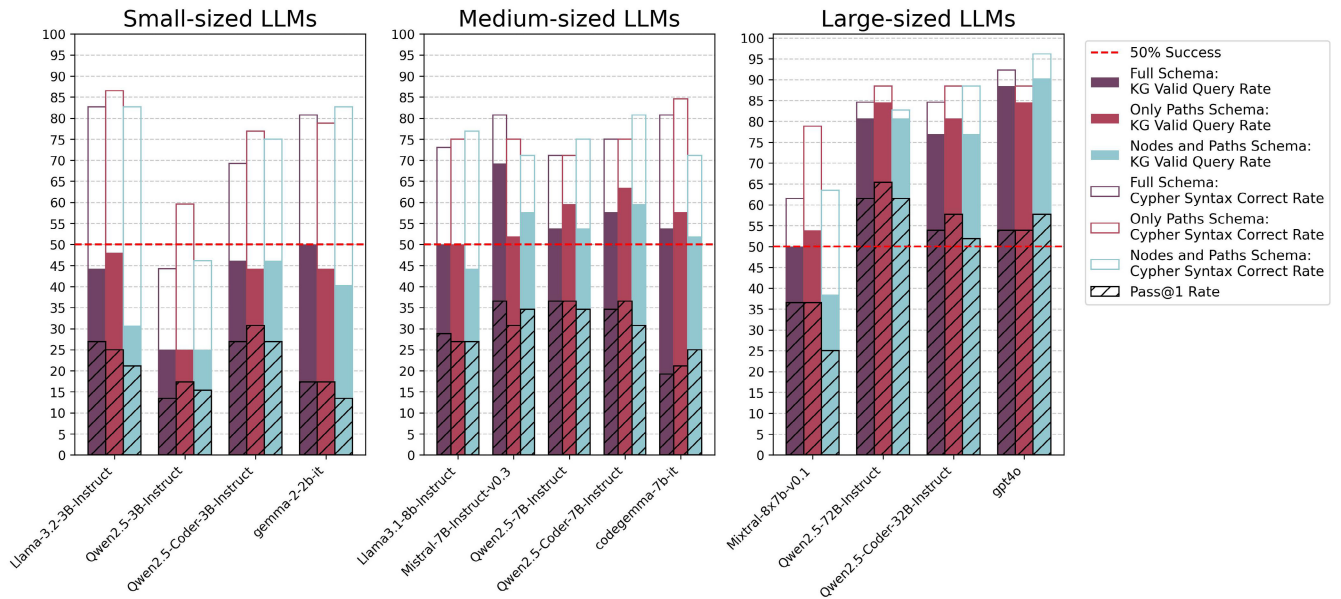


FIGURE 6. The KG Valid Query Rate, Cypher Syntax Correct Rate and Pass@1 Rate across the LLMs grouped by size.

Fig. 7. This figure presents the components of LLMetric and displays the LLMetric scores for the selected LLMs in descending order, using the optimal schema representation for each model. The plot highlights that models with a vast parameter space exhibit LLMetric score $> 65\%$, while the LLMetric score for medium-sized models ranges in (45%, 55%) and in small-sized models with more than 32 billion parameters, except the Mixtral-8 $\times$ 7b-v0.1 model, Qwen2.5-72B-Instruct model has slightly higher values in the key components of LLMetric, i.e. KG Valid Query Rate and Pass@1 Rate, compared with Qwen2.5-Coder-32B-Instruct model. However, Qwen2.5-Coder-32B-Instruct has higher adjustment factor values, which place it in a better ranking position and suggests that it may better improve responses to questions with incorrect output results with an enhanced instruction template format. Furthermore, Mixtral-8 $\times$ 7b-v0.1 model shows similar values across the components of the LLMetric and the LLMetric score compared to the Qwen2.5-7B-Instruct model, which performs slightly better and also is the best across all medium-sized models. Another interesting insight is that JaRou Factor is higher in small-sized models, while Jaccard Factor is higher in models with more than 7 billion parameters. This implies that small-sized models generate more Cypher queries closer to the expected Cypher queries, whereas medium-sized and large-sized models tend to retrieve output results more closely aligned with the expected ones. Finally, gpt4o maintains the first position across all metrics and composite ones, though with slight differences between each.

2) FAILURES REVIEW

For a comprehensive overview of each model's behavior in the Text-to-Cypher task, it is important to review the points

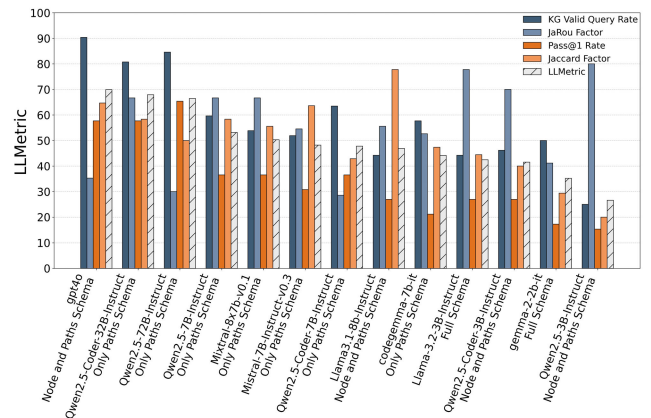


FIGURE 7. LLMetric and its components across the LLMs for the selected best schema representation.

of failure in the Review and Error Analysis stage. We consider as incorrect outputs the generated Cypher queries that failed to achieve $\text{pass}@1=\text{True}$, with correct or incorrect Cypher syntax. In Fig. 8 and Fig. 9 the types of questions regarding their complexity and depth reasoning level are thoroughly presented, in which each LLM has failed to generate a correct Cypher query. The results reveal a distinct pattern in model performance across different levels of question difficulty. Easy questions were largely handled correctly by all models, with Qwen2.5-Coder-32B-Instruct, Qwen2.5-72B-Instruct and gpt4o achieving the lowest errors rates, showcasing strong performance on basic Cypher clauses especially for larger models. In contrast, medium difficulty questions presented a greater challenge, with only the large-sized models, Qwen2.5-72B-Instruct and Qwen2.5-Coder-32B-Instruct exhibiting improved performance, reducing failure rates to below 65%. Despite these improvements,

a significant portion of small-sized and medium-sized models still struggled with slightly more advanced Cypher structure, using operators, basic sorting and aggregating clauses. The most notable challenge arises with hard questions that require advanced knowledge of the Cypher grammar. Qwen2.5-Coder-7B-Instruct and the large-sized models only managed to achieve success on some hard questions. This suggests that the model scale plays a crucial role in managing more intricate reasoning processes required for more challenging query types.

Fig. 9 reveals that most models struggle mainly in multi-hop questions, where more complex reasoning is required. However, large-sized models showcase advanced reasoning capabilities as were able to process multi-hop questions as effectively as simpler one-hop paths. Additionally, more than 50% of the single node access questions, despite their semantic simplicity, resulted in incorrect outputs in small-sized and medium-sized models, except for Qwen2.5-7B-Instruct and Qwen2.5-Coder-7B-Instruct, which were notably more successful. This indicates that even seemingly simple queries require a level of model sophistication that is beyond the reach of smaller-sized models.

D. OPTIMAL CONFIGURATION SETTINGS BASED ON PERFORMANCE CRITERIA

1) DECODING STRATEGIES COMPARISON

Based on the insights gained so far, the framework is applied to evaluate the LLMs under both greedy and random sampling decoding strategies, configured in the Prompt Engineering stage. The results indicate that neither strategy consistently outperformed the other, either across different models or within the same model, as the performance varied depending on the representation of the KG schema. The overall performance differences were minor, with LLMetric varying only by 0–5% between the two configurations within every model. Only Mixtral-8 × 7B-v0.1 exhibited a 10-20% performance improvement with random sampling compared to greedy strategy.

2) OVERALL PERFORMANCE OUTCOMES

After identifying for each LLM the optimal combination of instruction prompt, KG schema representation and decoding strategy, that maximizes its performance based on the LLMetric, we present the conclusive findings in Table 10.

The findings indicate that gpt4o, Qwen2.5-Coder-32B-Instruct, Qwen2.5-72B-Instruct achieved the highest performance rate, surpassing the other models by a significant margin. It should be noted that the state-of-the-art model gpt4o reveals only a 2% increase in performance compared to the open-source Qwen2.5-Coder-32B-Instruct model. The medium-sized models achieved a performance around 50%, with Qwen2.5-7B-Instruct surpassing the large-sized Mixtral-8 × 7B-v0.1 model. The small-sized models achieved the lowest LLMetric rates, validating that model size in

TABLE 10. The best decoding strategy, KG schema representation across LLMs based on the LLMetric.

Model	Configuration	Schema Representation	LLMetric
gpt4o	Sampling	Nodes and Paths Schema	69.93%
Qwen2.5-Coder-32B-Instruct	Sampling	Only Paths Schema	67.95%
Qwen2.5-72B-Instruct	Greedy	Only Paths Schema	67.95%
Qwen2.5-72B-Instruct	Greedy	Full Schema	67.12%
Qwen2.5-7B-Instruct	Sampling	Only Paths Schema	53.14%
Mixtral-8x7b-v0.1	Sampling	Only Paths Schema	50.28%
Qwen2.5-Coder-7B-Instruct	Greedy	Nodes and Paths Schema	48.22%
Mistral-7B-Instruct-v0.3	Sampling	Only Paths Schema	48.18%
Llama3.1-8b-Instruct	Greedy	Nodes and Paths Schema	46.88%
codegemma-7b-it	Sampling	Nodes and Paths Schema	46.88%
codegemma-7b-it	Sampling	Only Paths	44.16%
Llama-3.2-3B-Instruct	Sampling	Full Schema	42.44%
Qwen2.5-Coder-3B-Instruct	Greedy	Nodes and Paths Schema	41.54%
gemma-2-2b-it	Sampling	Nodes and Paths Schema	41.54%
gemma-2-2b-it	Greedy	Full Schema	36.96%
Qwen2.5-3B-Instruct	Greedy	Full Schema	36.96%
Qwen2.5-3B-Instruct	Greedy	Nodes and Paths Schema	28.85%

TABLE 11. Performance of medium-sized LLMs in Cypher query generation based on the LLMetric in 4-bit and 8-bit quantization.

Quant.	Model	Decoding Strategy	Schema Representation	LLMetric	Memory Usage	Inference Time
No	Qwen2.5-7B-Instruct	Sampling	Only Paths Schema	53.14%	—	—
	Qwen2.5-Coder-7B-Instruct	Greedy	Nodes and Paths Schema	48.22%	—	—
	Mistral-7B-Instruct-v0.3	Sampling	Only Paths Schema	48.18%	—	—
8-bit	Qwen2.5-7B-Instruct	Sampling	Only Paths Schema	49.04%	10.7 GB	21 min
	Qwen2.5-Coder-7B-Instruct	Greedy	Nodes and Paths Schema	48.57%	10.1 GB	15 min
	Mistral-7B-Instruct-v0.3	Sampling	Only Paths Schema	50.44%	13.2 GB	38 min
4-bit	Qwen2.5-7B-Instruct	Sampling	Only Paths Schema	48.37%	6.3 GB	7 min
	Qwen2.5-Coder-7B-Instruct	Greedy	Nodes and Paths Schema	47.5%	5.8 GB	5 min
	Mistral-7B-Instruct-v0.3	Sampling	Only Paths Schema	47.35%	11.4 GB	9 min

terms of parameters plays a significant role in the success of the Text-to-Cypher task. Considering this ranking of LLMs' performance, the Review and Error Analysis stage is crucial to reveal the type of errors in the Text-to-Cypher task for each model.

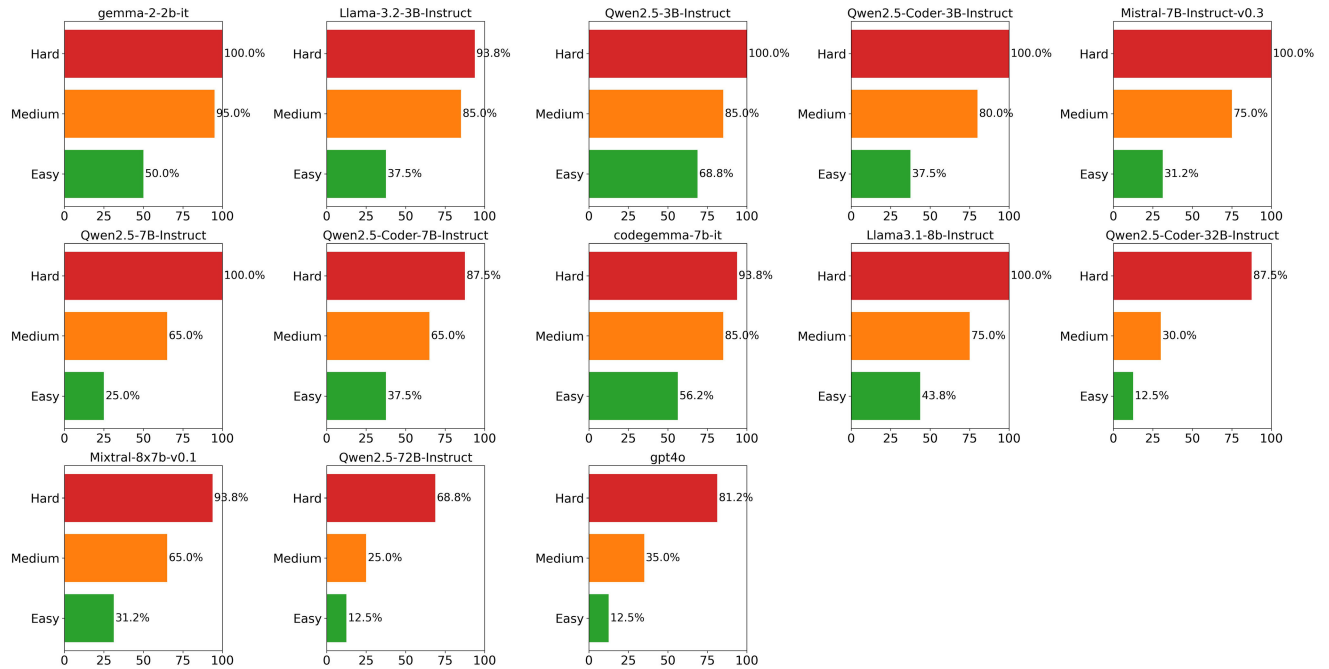


FIGURE 8. The percentage of questions with incorrect output result per complexity level.

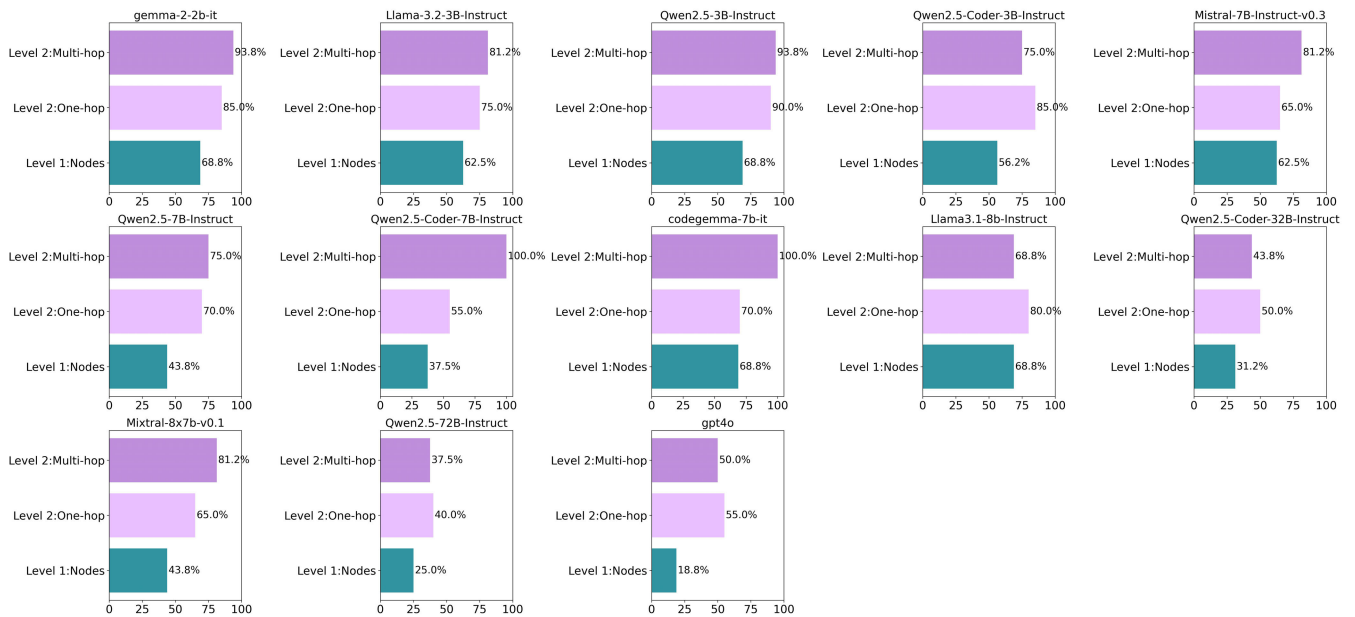


FIGURE 9. The percentage of questions with incorrect output result per depth reasoning level.

Regarding the schema representation, the small-sized models performed better with the Full Schema or Nodes and Paths Schema representation. This highlights a preference for more detailed KG schema descriptions over the more compact one that Only Paths schema offers. The medium-sized models, on the contrary, favored the smaller KG schema representations (Nodes and Paths, and Only Paths) and excluded the Full Schema. Notably, none of the open-source

large-sized models selected the Nodes and Paths schema, suggesting that a more comprehensive description of one-hop paths, including properties in both nodes and relationships, enhances their accuracy.

3) QUANTIZATION EFFECT ON PERFORMANCE

The medium-sized models for which we applied 4-bit and 8-bit quantization were the three models with the highest

performance in this group, namely Qwen2.5-7B-Instruct, Qwen2.5-Coder-7B-Instruct and Mistral-7B-Instruct-v0.3. As depicted in Table 11, quantization had minimal impact on the success rate of the models, with Qwen2.5-Coder-7B-Instruct and Mistral-7B-Instruct-v0.3 even showing slightly improved performance compared to their full-precision versions. 4-bit quantization, on the other side, has constantly the lowest performance across all models. Quantization significantly reduced the GPU memory footprint, allowing us to run these models locally on a 16GB GPU. The inference time for the input dataset of questions ranged from 15 to 38 minutes for the 8-bit models, and 5 to 9 minutes for the 4-bit models, highlighting the trade-off between inference speed and success rate.

E. PRIMARY SOURCES OF ERRORS

In the Review and Error Analysis stage, a thorough examination and analysis of errors of the LLMs in Table 10 is performed to identify the nature of their failures.

1) VALIDATOR-BASED PERFORMANCE ANALYSIS

The first phase of this stage aims to analyze exhaustively the Schema and Properties Validators performance of the selected LLMs. Fig. 10 and Fig. 11 showcase the number of syntactically correct generated queries categorized based on their Schema Validator and Properties Validator scores into the following value ranges: (0, 0.5], (0.5, 0.8], (0.8, 0.99], and (0.99, 1]. The first range (i.e., (0, 0.5]) represents the syntactically correct queries that do not align with the schema and properties of SustainGraph, while the second range (i.e., (0.5, 0.8]) depicts those that partially align with the schema. The third range (i.e., (0.8, 0.99]) includes almost correct Cypher queries that do not respect certain relationships, nodes, or properties. The final range (i.e., (0.99, 1]) represents the correctly generated queries.

By comparing the open-source models Qwen2.5-Coder-32B-Instruct and Qwen2.5-72B-Instruct with the gpt4o model, notable differences emerge in the level of respect to the KG schema and properties, despite their similar LLMetric scores. Gpt4o is the only model among the three that consistently adheres to the KG schema without hallucinating new nodes, relationships, or incorrect path directions. This is reflected in its achievement of a Schema Validator value of 1 across all syntactically correct generated queries in Fig. 10. Conversely, the large-sized Qwen2.5 models occasionally introduce structural errors, but their Schema Validator performance is always above 0.8.

However, when evaluating the accuracy of property usage in the generated Cypher queries, a different trend emerges. As shown in Fig. 11, Qwen2.5-72B-Instruct outperforms the other models, demonstrating superior reliability in accessing valid node and relationship properties. Notably, gpt4o exhibits a non-negligible failure rate in this aspect, with certain queries completely failing in accessing the correct properties (Properties Validator values in the range (0, 0.5]).

TABLE 12. A detailed explanation of the Error Type categories of queries.

Category	Description
Property Definition Value	This category covers Cypher queries that use undefined variables or parameters that are not properly defined. It is primarily related to properties, where the LLM incorrectly adds values like \$STRING or STRING as property values, based on the provided schema representation, resulting in incorrect Cypher syntax.
Cypher Logic Query	Generated queries that do not follow the logical structure of the Cypher language, with misplacement of clauses, incorrect use of logical operators. For example, these queries include the MATCH clause after the RETURN clause, do not use the AS keyword in the WITH clause, or place OR incorrectly inside the property value comparison, but not outside in the WHERE clause.
Invalid Cypher	Generated queries written using older Cypher syntax (i.e., functions or operators that are deprecated) or unsupported functions, leading to execution failures.
Schema/Properties Violation and Property Type Divergence	The generated queries that either do not adhere to the provided schema and properties or fail to access node and relationship properties correctly. For example, not using IN keyword to check a list property or not using date () function to access a property with type DATE.
Semantic Query	This category includes generated queries that are not semantically correct. These occur when the LLM misinterprets the meaning of the question, resulting in a query that does not accurately answer the intended question.
Result Structure Variation	This category consists of the generated queries that retrieve a result with different structure, i.e do not return the results in a list or access different properties of nodes or relationships compared to the ground-truth one.

As we examine medium-sized and small-sized models, it becomes apparent that a greater number of questions exhibit Schema and Properties validator values within lower validation ranges, of (0, 0.5] and (0.5, 0.8], as the size of the model decreases. These observations underscore the limitations of smaller-sized LLMs in respecting the provided KG schema, and therefore making them unsuitable for the Text-to-Cypher task.

2) CLASSIFICATION OF GENERATED CYPHER QUERY ERRORS

The second phase aims to inspect and categorize the failures of the aforementioned LLMs in syntactically incorrect generated queries and syntactically correct queries but with incorrect retrieved output (i.e. Jaccard output similarity score = 0). The set of syntactically incorrect generated Cypher queries was selected to assess vulnerabilities of the LLMs, while the syntactically correct set was chosen to examine the potential of the models for improving their output results. By reviewing the errors, we introduce six categories, which are thoroughly presented in Table 12. An incorrect Cypher query might belong to one or more categories of errors.

Fig. 12 illustrates the distribution of incorrect Cypher queries in the predefined error categories across dif-

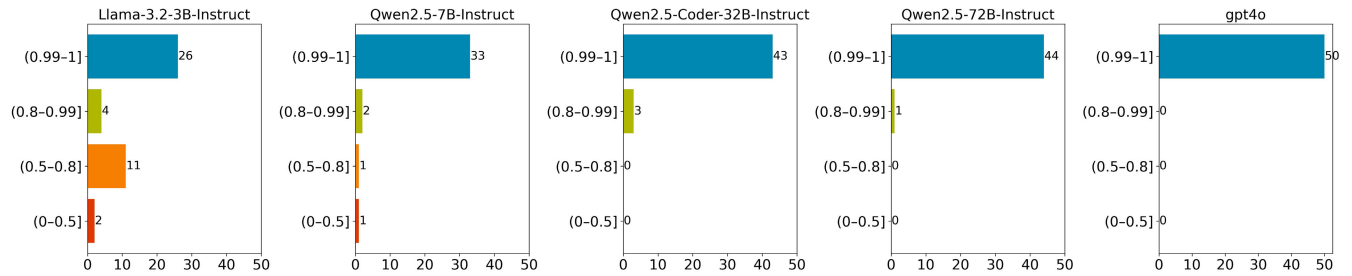


FIGURE 10. The distribution of syntactically correct generated queries across different Schema Validator value ranges in LLMs.

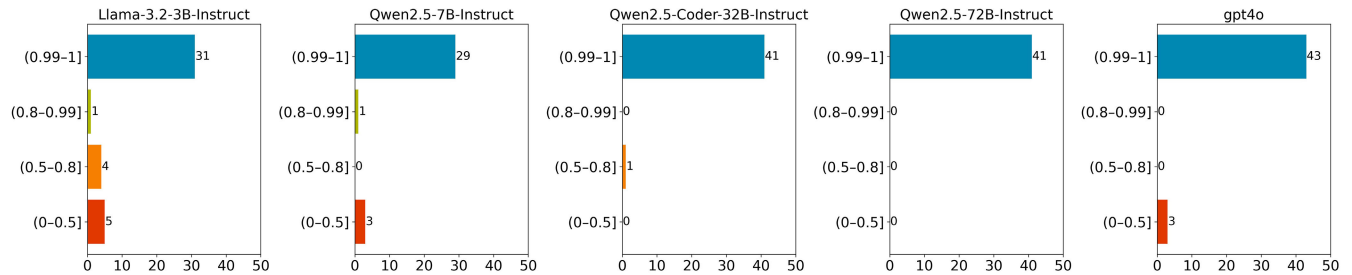


FIGURE 11. The distribution of syntactically correct generated queries across different Properties Validator value ranges in LLMs.

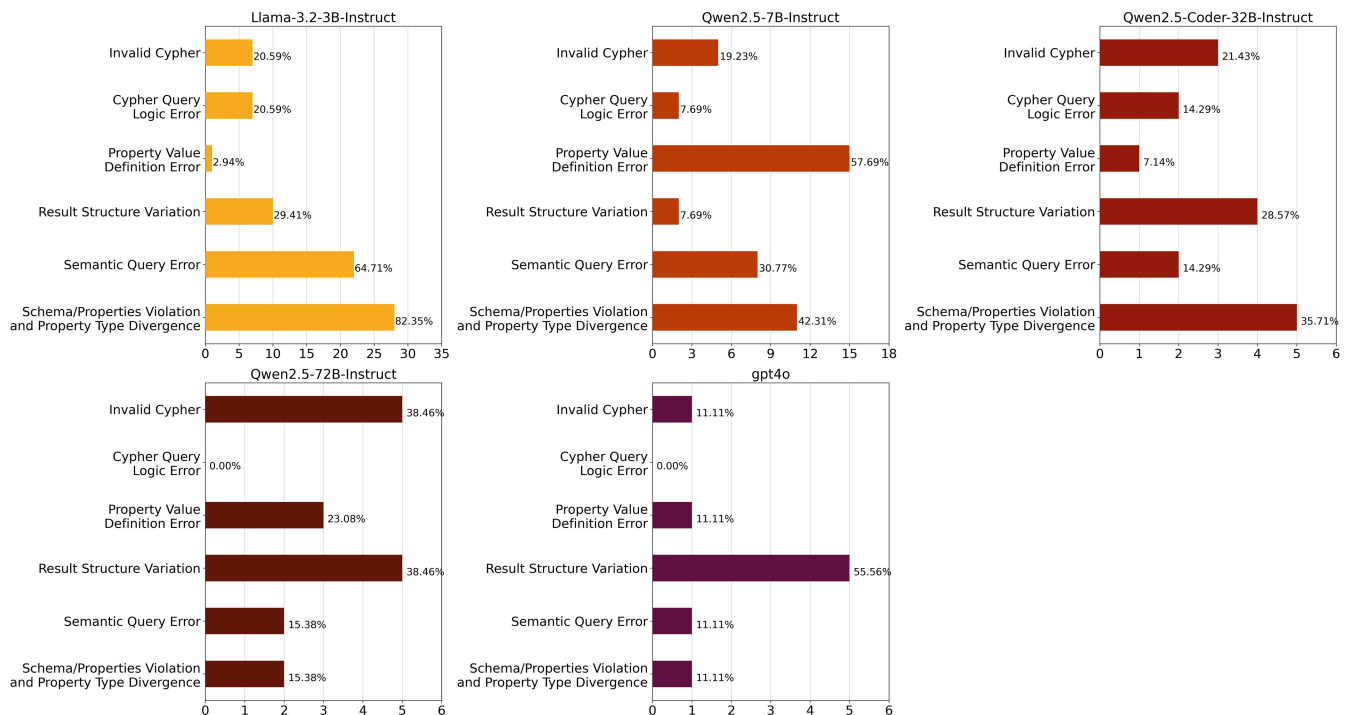


FIGURE 12. The distribution of the syntactically incorrect generated queries and syntactically correct queries but with incorrect output by the Error Types categories across the selected LLMs.

ferent models. Due to the potential overlap of queries across multiple categories, the percentage of incorrect queries associated with each category is displayed beside each bar.

The plots reveal some clear patterns correlating model size with error type distribution. As the parameter size increases, the Semantic Query error category rates decrease, indicating that smaller-sized LLMs struggle to comprehend the meaning

of the natural language question and express it through Cypher.

A similar decreasing trend is observed in the Schema/Properties Violation and Property Type Divergence category as model parameter space increases. However, for Qwen2.5-Coder-32B-Instruct, most errors in this error category stem from incorrect access of properties based on their type (i.e., Property Type Divergence) rather than Schema and Properties Violation, which are more common in small and medium-sized models, as evidenced by the high values of Schema and Properties Validator in Fig. 10 and 11.

Beyond this category, the Property Value Definition error is equally prevalent in the medium-sized Qwen2.5-7B-Instruct model, highlighting its tendency to set undefined variables or parameters as properties values. Similarly, larger LLMs, like Qwen2.5-72B-Instruct and gpt4o, also exhibit this type of error, further suggesting a challenge that persists across all model sizes.

In contrast, for the LLMs with larger parameter space, the most common errors in the generated queries belong to the Result Structure Variation error category. These errors do not fundamentally affect the information content retrieved from the query result, but rather they regard structural deviations in the way it is returned. Therefore, they are not critical in a downstream application and can easily be resolved, if needed, through prompt adjustments or explicit directions in the question.

It is notable that most errors generated by the gpt4o model belong to this category, while in the open-source Qwen models other error categories arise. Specifically, these models unexpectedly have a significant amount of Invalid Cypher errors, similar to the small and medium-sized ones, highlighting their lack of specific training in Cypher.

Finally, gpt4o and Qwen2.5-72B-Instruct are the only LLMs that did not generate Cypher Query Logic errors. Smaller-sized LLMs may occasionally struggle to correctly apply the logical structure of Cypher queries due to the reduced capacity to handle complex query logic. These findings highlight that each model struggles in different aspects in the Text-to-Cypher task, emphasizing that model's scale is not always a guarantee of robustness across all error types.

F. PROMPT REFINEMENT

This section focuses on prompt engineering techniques, which are the key to refining and optimizing the generation of Cypher queries. The prompt refinement process unfolds the ability of the best open-source selected model, Qwen2.5-Coder-32B-Instruct, to improve the generated results. Upon an extended review of the failures of the generated queries investigated in Section V-E, the errors related to the syntactically incorrect Cypher queries pertain to incorrect usage of DATE properties, undefined variables, invalid Cypher operators within property maps, and the inclusion of SQL operators. Conversely, the most errors of the syntactically correct generated queries with incorrect results

TABLE 13. Updated version of instructions in prompt template for Cypher query generation of Qwen2.5-Coder-32B-Instruct model.

Prompt instructions
You are an AI assistant designed to convert natural language questions into Cypher queries. Based on the Neo4j graph schema: {schema}, follow these rules: Write the query as plain text, no backticks, no explanations. LIST properties: Use UNWIND to extract values; use IN only for filtering. DATE properties: Extract components using dot notation (e.g. date.year). Format property values correctly based on their types. Define variables with a WITH clause before using them in property maps. Use the cypher operators like OR in only in WHERE clauses. Return only one valid Cypher query, adhering strictly to these rules.

TABLE 14. Comparison of LLMetric and its components across the initial and updated prompts of Qwen2.5-Coder-32B-Instruct model.

Prompt Version	KG Valid Query Rate	Pass@1 Rate	JaRou Factor	Jaccard Factor	LLMetric
First (Table 6)	80.8%	57.7%	66.7%	58.3%	67.9%
Updated (Table 13)	86.5%	65.4%	45.5%	72.7%	73.3%

(pass@1=False) belong to the Result Structure Variation error category. The most common failures in this category arise from not returning the properties declared in the question or failing to use appropriate clauses like UNWIND for properties of type LIST and COLLECT for grouping the results.

To address these failures, various instruction prompt templates are utilized to enhance model's performance. More extended prompts including explanations of instructions or instructions including "Avoid SQL operators, instead use Cypher clauses" seem to confuse the model's performance. Therefore, the instructions in the improved prompt in Table 13 are concise, positive and encouraging, ensuring adherence to them. The instructions depicted in Table 13 concern the proper management of properties' type, the definition of variables, and the right position of operators in a Cypher query.

Table 14 presents the LLMetric score and its components using the first and the updated version of the prompt template. An improvement of approximately 6% is observed in the LLMetric, reflecting an enhancement in the model performance. Although the improvement is modest, the two main components of the LLMetric, Pass@1 Rate and KG Valid Query Rate, improved by 8% and 6%, respectively, which are significant in the context of our study. Especially, a higher number of generated queries produced the exact same output as the ground-truth, as evidenced by the increase of Pass@1 Rate. Furthermore, the Jaccard Factor increased by 6%, indicating an improvement in the accuracy of the retrieved outputs. Regarding the JaRou Factor, a decrease is observed after the updated prompt. In the first version,

TABLE 15. Different types of KG schema description with examples.

KG Schema Description Type	Indicative Example in the SustainGraph
Full schema	<pre>Goal {title: STRING, description: STRING, code: STRING} Target {title: STRING, description: STRING, code: STRING} (:Goal {title: STRING, description: STRING, code: STRING})-[:HAS_TARGET]->(:Target {title: STRING, description: STRING, code: STRING}) (:CaseStudy {title: STRING, description: STRING, code: STRING})-[:ASSOCIATED_WITH {weight: INTEGER, description: STRING}]->(:Hazard {name: STRING, cluster: STRING, type: STRING})</pre>
Nodes and Paths	<pre>Goal {title: STRING, description: STRING, code: STRING} Target {title: STRING, description: STRING, code: STRING} (:Goal)-[:HAS_TARGET]->(:Target) (:CaseStudy)-[:ASSOCIATED_WITH {weight: INTEGER, description: STRING}]->(:Indicator)</pre>
Only paths	<pre>(:Goal {title: STRING, description: STRING, code: STRING})-[:HAS_TARGET]->(:Target {title: STRING, description: STRING, code: STRING}) (:CaseStudy {title: STRING, description: STRING, code: STRING})-[:ASSOCIATED_WITH {weight: INTEGER, description: STRING, cluster: STRING, type: STRING}]->(:Hazard {name: STRING, cluster: STRING, type: STRING})</pre>
Langchain schema	<pre>Node properties: Goal {title: STRING, description: STRING, code: STRING} Target {title: STRING, description: STRING, code: STRING} Relationship properties: ASSOCIATED_WITH {dateOfReport: DATE, relevance: STRING, climateResponse: STRING, weight: INTEGER, description: STRING} The relationships: (:Goal)-[:HAS_TARGET]->(:Target) (:CaseStudy)-[:ASSOCIATED_WITH {weight: INTEGER, (:CaseStudy)-[:ASSOCIATED_WITH {weight: INTEGER, description: STRING}]->(:Indicator)</pre>

its value was higher, indicating that 66.7% of the KG Valid queries that retrieved incorrect output (pass@1=False) had JaRou values above 0.8. This suggested that small adjustments to the Cypher structure could potentially lead to successful results. With the updated tailored prompt, most of these queries achieved a pass@1=True as validated by the increase of Pass@1 Rate, leading to a decrease in the JaRou Factor. A closer examination of the KG Valid generated

queries with pass@1=False indicated that their JaRou scores were around 0.7, suggesting that further adjustments could improve their performance.

Overall, the results demonstrate an improvement of the best selected model's performance by tailoring the instructions. In the context of our study, the evaluation criteria are more strict with regard to the result structure, as we evaluate the performance of models given a clear and accurate question that reflects the key components that the Cypher query may return. A greater improvement can be achieved by incorporating additional clues into the questions, such as "unique" to enable the use of DISTINCT, or "grouping them" to leverage COLLECT clause. The decision not to implement this approach was made in order to simulate a user who is familiar with the knowledge graph schema, but lacks a technical background. A more lenient evaluation could lead to higher improvements in Table 13.

VI. CONCLUSION AND FUTURE WORK

Converting natural language to Cypher query language can be a challenging task for LLMs, especially when dealing with complex KG schemas. Assessing the performance of various open-source LLMs in the Text-to-Cypher task over the SustainGraph revealed that open-source larger-sized models outperform smaller-sized and medium-sized ones. Especially, the open-source Qwen models with a vast parameter space attained the exact same output as the ground-truth for more than half of the questions, with over 80% of the Cypher queries generated being syntactically correct and adhering to the provided schema. Our study also revealed that all models struggle to answer multi-hops questions or questions of medium/hard complexity, with the larger-sized ones achieving again better success on some of them. Contrary to expectations, the results showed that the performance of the Mixtral-8 × 7b-v0.1 model, which belongs to the large-sized models, was comparable to that of the medium-sized models. Conversely, some medium-sized and smaller-sized models performed similarly to larger-sized ones in Cypher syntax, but remained unreliable overall. Through the in-depth review and error analysis, our study shows that, despite the fact that the gpt4o model performed slightly better across the evaluation metrics compared to the open-source large-sized models, most of the errors are attributed to the structure of the retrieved result. Through strategic prompt engineering, the performance of the best open-source model was optimized. Despite these advancements, open-source LLMs still encounter difficulties with Cypher logic, and the instructions used in prompt refinement were tailored specifically to the questions of indicative dataset. Therefore, developing a universal prompt for all Cypher logic cases is impossible. Further fine-tuning of models on Cypher and the development of new approaches to better integrate KG schemas into the generation process remain an important area of research that requires further investigation.

Considering that evaluating the performance of LLMs in the Text-to-Cypher task is multi-aspect, the proposed

evaluation framework provides a systematic method, utilizing comprehensive metrics and validators. These validators, offered as an open-source Python library, the CyVer library, may be utilized not only for the evaluation of LLMs in Cypher generation, but also as a baseline for agentic AI workflows that refine a generated Cypher query. Selecting a reliable LLM for the Text-to-Cypher task, however, remains a non-trivial challenge. We demonstrate the weaknesses and strengths of the selected LLMs, particularly the open-source models that require limited computation resources or cost-free API access. By consolidating these evaluation practices into a unified framework, our work provides a baseline performance evaluation in the applications of LLMs with KGs. The challenge of selecting the appropriate LLM is addressed by comparing the performance of several models, aiding in the selection of the most suitable option for specific applications.

Although this study provides valuable insights on the performance of open-source LLMs in the Text-to-Cypher task, it should be noted that this task remains subject to computational limitations. Evaluating LLMs with limited computational resources presents limitations, including restricted model size, inability to fine-tune larger-sized models, or even memory constraints for real-time applications. The selection of LLMs was based on the constraints of 16GB GPU or cost-effective API access. Furthermore, the models were evaluated using a dataset of 52 rigorous questions based on the SustainGraph. Each question explicitly included the exact names of node labels and relationship types and all relevant one-hop paths were clearly mentioned to ensure comprehensive alignment with the KG schema. However, a broader evaluation involving more generic and challenging questions—incorporating synonyms, semantic variability, and inferred (implicit) paths—would better reflect real-world user queries and further test the robustness of LLMs in the Text-to-Cypher task.

Several key directions for future research emerge from this study, including simpler ones, such as few-shot prompt techniques, to more advanced ones like fine-tuning of the models for this task. Fine-tuning LLMs for this task demands significant computational power and a substantial volume of training data. Future directions may include the exploration of hybrid techniques that combine prompt engineering techniques, presented in this study, with query refinement agents that address incorrect Cypher logic. These refined strategies may also be evaluated on a dataset with more sophisticated questions. Furthermore, Named Entity Recognition (NER) techniques could also be helpful for the extraction of main entities in the user's questions that could be provided to the prompt template of the QA system, helping the LLMs to increase their accuracy and reliability.

APPENDIX A

KG SCHEMA DESCRIPTION TYPES

Table 15 shows an example of each type of the SustainGraph schema description.

REFERENCES

- [1] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, Jan. 2024.
- [2] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2020, pp. 1877–1901.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [4] J. Lee, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H., T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent abilities of large language models," *Trans. Mach. Learn. Res.*, Jan. 2022.
- [5] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [6] N. Chakraborty, D. Lukovnikov, G. Maheshwari, P. Trivedi, J. Lehmann, and A. Fischer, "Introduction to neural network-based question answering over knowledge graphs," *WIREs Data Mining Knowl. Discovery*, vol. 11, no. 3, p. 1389, May 2021.
- [7] J. Baek, A. F. Aji, and A. Saffari, "Knowledge-augmented language model prompting for zero-shot knowledge graph question answering," in *Proc. 1st Workshop Natural Lang. Reasoning Structured Explanations (NLRSE)*, Toronto, ON, Canada, B. D. Mishra, G. Durrett, P. Jansen, D. Neves Ribeiro, and J. Wei, Eds., 2023, pp. 78–106.
- [8] B.-H. Yoon, S.-K. Kim, and S.-Y. Kim, "Use of graph database for the integration of heterogeneous biological data," *Genomics Informat.*, vol. 15, no. 1, p. 19, 2017.
- [9] J. Guo, L. Du, and H. Liu, "GPT4Graph: Can large language models understand graph structured data? An empirical evaluation and benchmarking," 2023, *arXiv:2305.15066*.
- [10] J. Jiang, K. Zhou, Z. Dong, K. Ye, X. Zhao, and J.-R. Wen, "StructGPT: A general framework for large language model to reason over structured data," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Singapore, 2023, pp. 9237–9251.
- [11] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-SQL," *Vldb J.*, vol. 32, no. 4, pp. 905–936, Jul. 2023.
- [12] A. Guo, X. Zhao, and W. Ma, "ER-SQL: Learning enhanced representation for text-to-SQL using table contents," *Neurocomputing*, vol. 465, pp. 359–370, Nov. 2021.
- [13] C. V. S. Avila, V. M. P. Vidal, W. Franco, and M. A. Casanova, "Experiments with text-to-SPARQL based on ChatGPT," in *Proc. IEEE 18th Int. Conf. Semantic Comput. (ICSC)*, Feb. 2024, pp. 277–284.
- [14] S. Purkayastha, S. Dana, D. Garg, D. Khandelwal, and G. P. S. Bhargava, "Knowledge graph question answering via SPARQL silhouette generation," 2021, *arXiv:2109.09475*.
- [15] M. Hornsteiner, M. Kreussel, C. Steindl, F. Ebner, P. Empl, and S. Schöning, "Real-time text-to-cypher query generation with large language models for graph databases," *Future Internet*, vol. 16, no. 12, p. 438, Nov. 2024.
- [16] S. Sivasubramaniam, C. Osei-Akoto, Y. Zhang, K. Stockinger, and J. Fuerst, "SM3-text-to-query: Synthetic multi-model medical text-to-query benchmark," in *Proc. Adv. Neural Inf. Process. Syst.*, Nov. 2024, pp. 88627–88663.
- [17] M. G. Ozsoy, L. Messallem, J. Besga, and G. Minneci, "Text2Cypher: Bridging natural language and graph databases," 2024, *arXiv:2412.10064*.
- [18] E. Fotopoulou, I. Mandilara, A. Zafeiropoulos, C. Laspidou, G. Adamos, P. Koundouri, and S. Papavassiliou, "SustainGraph: A knowledge graph for tracking the progress and the interlinking among the sustainable development goals' targets," *Frontiers Environ. Sci.*, vol. 10, Oct. 2022.
- [19] SustainGraph LLM Evaluation Gitlab Repository. (2025). C. M. Androna, I. Mandilara, E. Fotopoulou, A. Zafeiropoulos, SustainGraph LLM Evaluation Gitlab Repository. [Online]. Available: https://gitlab.com/netmode/sustaingraph_llm_evaluation
- [20] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Brussels, Belgium, 2018, pp. 3911–3921.
- [21] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSL: Decoupling schema linking and skeleton parsing for text-to-SQL," in *Proc. 37th AAAI Conf. Artif. Intell. 35th Conf. Innov. Appl. Artif. Intell. 13th Symp. Educ. Adv. Artif. Intell.*, 2023.

- [22] Y. Gan, X. Chen, and M. Purver, "Exploring underexplored limitations of cross-domain text-to-SQL generalization," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2021, pp. 8926–8931.
- [23] Y. Gan, X. Chen, J. Xie, M. Purver, J. R. Woodward, J. Drake, and Q. Zhang, "Natural SQL: Making SQL easier to infer from natural language specifications," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, M.-F. Moens, X. Huang, L. Specia, and S. W.-T. Yih, Eds., Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 2030–2042.
- [24] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-grounded pretraining for text-to-SQL," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2021, pp. 1337–1350.
- [25] V. W. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," 2017, *arXiv:1709.00103*.
- [26] C.-H. Lee, O. Polozov, and M. Richardson, "KaggleDBQA: Realistic evaluation of text-to-SQL parsers," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process. (Long Papers)*, vol. 1, 2021, pp. 2261–2273.
- [27] T. Yu et al., "CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases," 2019, *arXiv:1909.05378*.
- [28] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, C. Ma, K. C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A BIG bench for large-scale database grounded text-to-SQLs," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2023, pp. 42330–42357.
- [29] Z. Gu, J. Fan, N. Tang, L. Cao, B. Jia, S. Madden, and X. Du, "Few-shot text-to-SQL translation using structure and content prompt learning," *Proc. ACM Manage. Data*, vol. 1, no. 2, pp. 1–28, Jun. 2023.
- [30] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL empowered by large language models: A benchmark evaluation," *Proc. VLDB Endowment*, vol. 17, no. 5, pp. 1132–1145, Jan. 2024.
- [31] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2023, pp. 36339–36348.
- [32] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, Q. Zhang, Y. Zhao, and Z. Li, "MAC-SQL: A multi-agent collaborative framework for text-to-SQL," in *Proc. 31st Int. Conf. Comput. Linguistics*, Jan. 2023, pp. 540–557.
- [33] E. R. Nascimento, G. Garcia, Y. T. Izquierdo, L. Feijó, G. M. C. Coelho, A. R. de Oliveira, M. Lemos, R. L. S. Garcia, L. A. P. P. Leme, and M. A. Casanova, "LLM-based text-to-SQL for real-world databases," *Social Netw. Comput. Sci.*, vol. 6, no. 2, p. 130, Jan. 2025.
- [34] L. Dong and M. Lapata, "Language to logical form with neural attention," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (Long Papers)*, vol. 1, Berlin, Germany, 2016, pp. 33–43.
- [35] X. Yin, D. Gromann, and S. Rudolph, "Neural machine translating from natural language to SPARQL," *Future Gener. Comput. Syst.*, vol. 117, pp. 510–519, Apr. 2021.
- [36] P. Trivedi, G. Maheshwari, M. Dubey, and J. Lehmann, "LC-QuAD: A corpus for complex question answering over knowledge graphs," in *Proc. 16th Int. Semantic Web Conf.*, Vienna, Austria, Jan. 2017, pp. 210–218.
- [37] M. Dubey, D. Banerjee, A. Abdelkawi, and J. Lehmann, "LC-QuAD 2.0: A large dataset for complex question answering over Wikidata and dbpedia," in *The Semantic Web—ISWC 2019*, C. Ghidini, O. Hartig, M. Maleshkova, V. Svátek, I. Cruz, A. Hogan, J. Song, M. Lefrançois, and F. Gandon, Eds., Cham, Switzerland: Springer, 2019, pp. 69–78.
- [38] A.-K. Hartmann, E. Marx, and T. Soru, "Generating a large dataset for neural question answering over the DBpedia knowledge base," in *Proc. Workshop Linked Data Manag., Co-Located With W3C (WEBDB)*, 2018.
- [39] C. Kosten, P. Cudré-Mauroux, and K. Stockinger, "Spider4SPARQL: A complex benchmark for evaluating knowledge graph question answering systems," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2023, pp. 5272–5281.
- [40] M. Ben Amor, A. Strappazzon, M. Granitzer, E. Egyed-Zsigmond, and J. Mitrović, "Instruct-to-SPARQL: A text-to-SPARQL dataset for training SPARQL agents," in *Proc. ACM SIGIR Conf. Human Inf. Interact. Retr.*, Melbourne, VIC, Australia, Mar. 2025, pp. 390–395.
- [41] F. F. Luz and M. Finger, "Semantic parsing natural language into SPARQL: Improving target language representation with neural attention," 2018, *arXiv:1803.04329*.
- [42] R. Omar, I. Dhall, P. Kalnis, and E. Mansour, "A universal question-answering platform for knowledge graphs," *Proc. ACM Manage. Data*, vol. 1, no. 1, pp. 1–25, May 2023.
- [43] A. Guo, X. Li, G. Xiao, Z. Tan, and X. Zhao, "SpCQL: A semantic parsing dataset for converting natural language into cypher," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, New York, NY, USA, Oct. 2022, pp. 3973–3977.
- [44] Z. Zhao, W. Liu, T. French, and M. Stewart, "CySpider: A neural semantic parsing corpus with baseline models for property graphs," in *AI 2023: Advances in Artificial Intelligence*, T. Liu, G. Webb, L. Yue, and D. Wang, Eds., Singapore: Springer, 2024, pp. 120–132.
- [45] Z. Zhong, L. Zhong, Z. Sun, Q. Jin, Z. Qin, and X. Zhang, "SyntheT2C: Generating synthetic data for fine-tuning large language models on the Text2Cypher task," 2024, *arXiv:2406.10710*.
- [46] Y. Zhou, Y. He, S. Tian, Y. Ni, Z. Yin, X. Liu, C. Ji, S. Liu, X. Qiu, G. Ye, and H. Chai, "R3-NL2GQL: A model coordination and knowledge graph alignment approach for NL2GQL," in *Proc. Findings Assoc. Comput. Linguistics (EMNLP)*, Miami, FL, USA, 2024, pp. 13679–13692.
- [47] Y. Liang, T. Xie, G. Peng, Z. Huang, Y. Lan, and W. Qian, "NAT-NL2GQL: A novel multi-agent framework for translating natural language to graph query language," 2024, *arXiv:2412.10434*.
- [48] Y. Feng, S. Papicchio, and S. Rahman, "CypherBench: Towards precise retrieval over full-scale modern knowledge graphs in the LLM era," 2024, *arXiv:2412.18702*.
- [49] CyVer Gitlab Repository. (2025). I. Mandilara, C. M. Androna, E. Fotopoulou, A. Zafeiropoulos, CyVer Gitlab Repository. [Online]. Available: <https://gitlab.com/netmode/CyVer>
- [50] C.-M. Androna, I. Mandilara, E. Fotopoulou, K. Giannakopoulou, A. Zafeiropoulos, and S. Papavassiliou, "Socio-environmental spatial data reduction and integration in knowledge graphs," in *Proc. Panhellenic Conf. Electron. Telecommun. (PACET)*, Thessaloniki, Greece, Mar. 2024, pp. 1–6.
- [51] C. M. Androna, I. Mandilara, E. Fotopoulou, A. Zafeiropoulos, and S. Papavassiliou, "A knowledge graph-driven analysis of the interlinkages among the sustainable development goal indicators in different spatial resolutions," *Sustainability*, vol. 16, no. 11, p. 4328, May 2024.
- [52] I. Mandilara, "Knowledge graph data enrichment based on a software library for text mapping to the sustainable development goals," in *Proc. TEXT2KG 2nd Int. Workshop Knowl. Graph Gener. From Text Co-Located With Extended Semantic Web Conf. (ESWC)*, Hersionissos, Greece, 2023.
- [53] ARSINOE Project. (2023). ARSINOE H2020 Project: Climate Resilient Regions Through Systemic Solutions and Innovations. [Online]. Available: <https://arsinoe-project.eu/>
- [54] S. Zhao, Y. Yang, Z. Wang, Z. He, L. K. Qiu, and L. Qiu, "Retrieval augmented generation (RAG) and beyond: A comprehensive survey on how to make your LLMs use external data more wisely," 2024, *arXiv:2409.14924*.
- [55] Neo4j Cypher Integration, LangChain, San Francisco, CA, USA, 2024. Accessed: Jan. 30, 2024.
- [56] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, Jan. 2022.



IOANNA MANDILARA received the bachelor's degree in applied mathematical and physical sciences from the National Technical University of Athens (NTUA), and the master's degree in data science and machine learning from the Interdepartmental Program of Postgraduate Studies (Master's Degree Program), NTUA. She is currently working in research and development projects as a Research Assistant with the NETWORK Management and Optimal DEsign Laboratory, School of Electrical and Computer Engineering, NTUA. Previously, she was a Data Scientist in various company projects. Her main research interests focus on the modeling and development of knowledge graphs, big data management, mathematical modeling, and the development of deep machine learning models. She is skilled in employing natural language processing techniques and reinforcement learning algorithms for in-depth data analysis. She has participated in publications in international journals and conferences.



CHRISTINA MARIA ANDRONA received the bachelor's and integrated master's degrees in electrical and computer engineering from the National Technical University of Athens (NTUA), where she is currently pursuing the Ph.D. degree in computer science with the School of Electrical and Computer Engineering. Her work involves developing scalable models for data integration, optimization, and cross-domain extraction, with a strong emphasis on evaluation methodologies. She has contributed to EU research projects with the NETWORK Management and Optimal DDesign Laboratory. She has co-authored publications in peer-reviewed conferences and journals. Her research focuses on knowledge graphs, machine learning, natural language processing, and generative artificial intelligence.



ELENI FOTOPOULOU received the degree in electrical and computer engineering from the National Technical University of Athens, in 2009, the degree in therapeutic pedagogy from the University of Thessaly, in 2013, and the master's degree in emotional intelligence from the University of Barcelona, in 2018. She is currently pursuing the Ph.D. degree with the NETMODE Laboratory, National Technical University of Athens. She is a Senior Software Engineer with expertise on the design and development of data management and analytics solutions in various domains, including educational, environmental, and cloud computing domains. She has been involved in a series of EC-funded research projects. Her research interests include cloud-applications, intelligent orchestration, data mining techniques, semantic technologies, deep reinforcement learning, psychometry/sociometry, and emotional intelligence.



ANASTASIOS ZAFEIROPOULOS (Member, IEEE) received the Dipl.-Ing. and Ph.D. degrees from the School of Electrical and Computer Engineering, National Technical University of Athens, and the M.Sc. degree in public policy and management from Athens University of Economics and Business. He is currently a Post-doctoral Researcher with the School of Electrical and Computer Engineering, National Technical University of Athens. He has participated in multiple EU and national funded research and innovation projects, focusing mainly on the design and implementation of innovative technical solutions on the fields of education, environmental sciences, the IoT, and energy-efficient IT technologies.



SYMEON PAPAVALASSILIOU (Senior Member, IEEE) is currently a Professor with the School of Electrical and Computer Engineering, National Technical University of Athens (NTUA). From 1995 to 1999, he was a Senior Technical Staff Member with AT&T Laboratories, NJ, USA. In August 1999, he joined the ECE Department, New Jersey Institute of Technology, USA, where he was an Associate Professor, until 2004. He has an established record of publications, while he has received several scientific awards and distinctions. His research interests include the areas of optimization and performance evaluation of distributed and complex systems.

...